EX 3 : COMPARISON OF SGD WITH MOMENTUM VS ADAM OPTIMIZER

AIM:

To Implement a training algorithm using Stochastic Gradient Descent (SGD) with momentum and compare it with the Adam optimizer. Train both models on a dataset and compare their convergence rates and performance.

OBJECTIVES:

1. Understand the principles of optimization algorithms in deep learning..
2. Implement and train models using SGD with momentum and Adam.
3. Analyze and compare the learning behavior and convergence patterns of the two optimizers.
4. Visualize loss and accuracy across epochs for both optimization methods.

ALGORITHM:

1. Import the required libraries, including PyTorch, torchvision, and Matplotlib.
2. Select the computing device (GPU or CPU) and load the CIFAR-10 dataset.
3. Preprocess the images by converting them into tensors and normalizing pixel values.
4. Create a DataLoader to load training images in batches of 128 and shuffle the data.
5. Define a Multi-Layer Perceptron (MLP) with an input layer, a hidden layer containing 256 neurons with ReLU activation, and an output layer with 10 neurons.
6. Define the training function using cross-entropy loss and record the loss and accuracy for each epoch.
7. Initialize the first MLP model and configure the SGD optimizer with a learning rate of 0.01 and momentum of 0.9.
8. Train the first model by performing forward propagation, calculating loss, backpropagation, and updating the model parameters using SGD with momentum.
9. Initialize a second MLP model and configure the Adam optimizer with a learning rate of 0.001.
10. Train the second model using the same dataset and training procedure with the Adam optimizer.
11. Record the training loss and accuracy of both models across epochs.
12. Plot the loss and accuracy curves for both optimizers and compare their convergence rates and training performance.
13. Analyze the results to determine which optimizer achieves faster convergence and better training accuracy.

SOURCE CODE:

```
# Step 1: Import Necessary Libraries
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Step 2: Set Device and Load Dataset
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)

transform = transforms.Compose([
   transforms.ToTensor(),
   transforms.Normalize((0.5, 0.5, 0.5),
                        (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
   root='./data',
   train=True,
   transform=transform,
   download=True
)

trainloader = torch.utils.data.DataLoader(
   trainset,
   batch_size=128,
   shuffle=True
)

# Step 3: Define MLP Model
class MLP(nn.Module):
   def __init__(self):
      super().__init__()

      self.flatten = nn.Flatten()

      self.net = nn.Sequential(
         nn.Linear(3 * 32 * 32, 256),
         nn.ReLU(),
         nn.Linear(256, 10)
      )
```

```
    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)

# Step 4: Define Training Function
def train(model, optimizer, epochs=10):

    model.to(device)

    loss_fn = nn.CrossEntropyLoss()

    losses = []
    accuracies = []

    for epoch in range(epochs):

        total_loss = 0
        correct = 0
        total = 0

        model.train()

        for imgs, labels in trainloader:

            imgs = imgs.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(imgs)

            # Calculate loss
            loss = loss_fn(outputs, labels)

            # Backward pass
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            # Calculate loss
            total_loss += loss.item()

            # Calculate accuracy
            _, predicted = outputs.max(1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()
```

```
        # Average loss and accuracy
        avg_loss = total_loss / len(trainloader)
        accuracy = 100.0 * correct / total

        losses.append(avg_loss)
        accuracies.append(accuracy)

        print(
            f"Epoch {epoch + 1}: "
            f"Loss = {avg_loss:.4f}, "
            f"Accuracy = {accuracy:.2f}%"
        )

    return losses, accuracies

# Step 5: Train with SGD + Momentum
print("\n===== SGD WITH MOMENTUM =====")

model_sgd = MLP()

sgd = optim.SGD(
    model_sgd.parameters(),
    lr=0.01,
    momentum=0.9
)

losses_sgd, acc_sgd = train(
    model_sgd,
    sgd,
    epochs=10
)

# Step 5: Train with Adam
print("\n===== ADAM OPTIMIZER =====")

model_adam = MLP()

adam = optim.Adam(
    model_adam.parameters(),
    lr=0.001
)

losses_adam, acc_adam = train(
    model_adam,
    adam,
```

```
    epochs=10
)

# Step 6: Visualize Loss and Accuracy Comparison

plt.figure(figsize=(12, 5))

# Loss Comparison
plt.subplot(1, 2, 1)

plt.plot(
    range(1, 11),
    losses_sgd,
    marker='o',
    label="SGD + Momentum"
)

plt.plot(
    range(1, 11),
    losses_adam,
    marker='o',
    label="Adam"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Comparison on CIFAR-10")
plt.legend()
plt.grid(True)

# Accuracy Comparison
plt.subplot(1, 2, 2)

plt.plot(
    range(1, 11),
    acc_sgd,
    marker='o',
    label="SGD + Momentum"
)

plt.plot(
    range(1, 11),
    acc_adam,
    marker='o',
    label="Adam"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison on CIFAR-10")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# Final Results
print("\n===== FINAL RESULTS =====")

print(
    f"SGD + Momentum - "
    f"Final Loss: {losses_sgd[-1]:.4f}, "
    f"Final Accuracy: {acc_sgd[-1]:.2f}%"
)

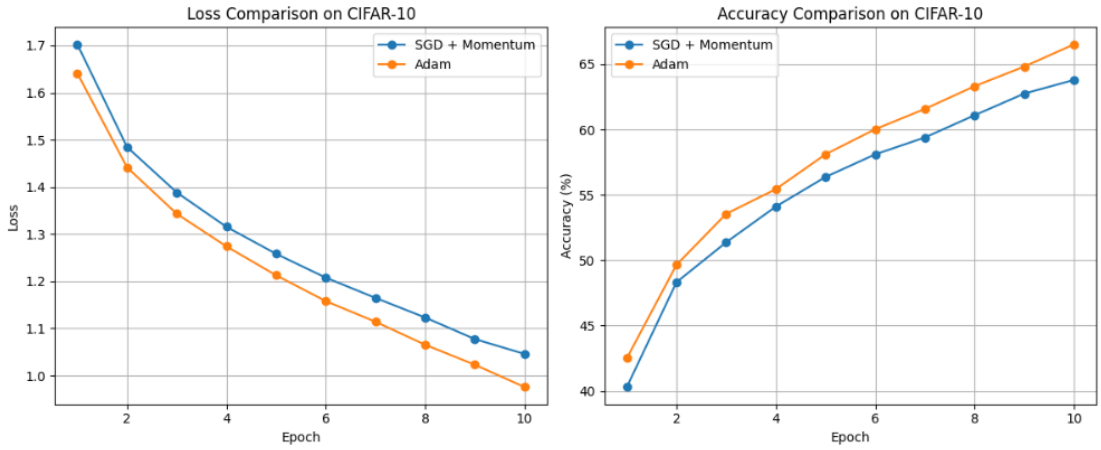
print(
    f"Adam - "
    f"Final Loss: {losses_adam[-1]:.4f}, "
    f"Final Accuracy: {acc_adam[-1]:.2f}%"
)
```

OUTPUT:

```
Device: cpu
100%|██████████| 170M/170M [04:22<00:00, 650kB/s]

===== SGD WITH MOMENTUM =====
Epoch 1: Loss = 1.7024, Accuracy = 40.30%
Epoch 2: Loss = 1.4847, Accuracy = 48.32%
Epoch 3: Loss = 1.3884, Accuracy = 51.37%
Epoch 4: Loss = 1.3155, Accuracy = 54.10%
Epoch 5: Loss = 1.2582, Accuracy = 56.37%
Epoch 6: Loss = 1.2077, Accuracy = 58.10%
Epoch 7: Loss = 1.1647, Accuracy = 59.39%
Epoch 8: Loss = 1.1231, Accuracy = 61.09%
Epoch 9: Loss = 1.0774, Accuracy = 62.74%
Epoch 10: Loss = 1.0459, Accuracy = 63.79%

===== ADAM OPTIMIZER =====
Epoch 1: Loss = 1.6411, Accuracy = 42.50%
Epoch 2: Loss = 1.4415, Accuracy = 49.65%
Epoch 3: Loss = 1.3431, Accuracy = 53.55%
Epoch 4: Loss = 1.2740, Accuracy = 55.44%
Epoch 5: Loss = 1.2126, Accuracy = 58.10%
Epoch 6: Loss = 1.1579, Accuracy = 60.02%
Epoch 7: Loss = 1.1141, Accuracy = 61.57%
Epoch 8: Loss = 1.0653, Accuracy = 63.31%
Epoch 9: Loss = 1.0229, Accuracy = 64.81%
Epoch 10: Loss = 0.9757, Accuracy = 66.51%
```




```
===== FINAL RESULTS =====
SGD + Momentum - Final Loss: 1.0459, Final Accuracy: 63.79%
Adam - Final Loss: 0.9757, Final Accuracy: 66.51%
```

RESULT:

This experiment shows a practical implementation of two optimization techniques and highlights their impact on model performance. Adam typically converges faster due to adaptive learning rates, while SGD with momentum offers more controlled, gradual learning. Choosing the right optimizer depends on the nature of the data and the task complexity.